MMDU MULLANA OFFICIAL SYLLABUS • BCSE-004 • UNIT 4

Unit 4: File Handling & Object-Oriented Programming (OOP in Python)

Subject: Python Programming (BCSE-004) • **Exam Weightage:** 26 - 30 Marks • **Study Time:** 40 mins

File streams, 9-tier access modes, context managers (with open), pointer navigation (tell/seek), OOP classes, self parameter, class vs instance variables, dunder methods, operator overloading, inheritance hierarchies, MRO (C3 Linearization), polymorphism, and composition.

1. File Handling Streams & Context Managers

1. Complete 9-Tier File Access Modes Matrix

Mode	Initial Cursor Position	File Existence Requirement	Truncation Behavior
`'r'`	Beginning (offset 0)	**File must exist** (raises `FileNotFoundError`)	No truncation
`'w'`	Beginning (offset 0)	Created automatically if absent	**Immediate truncation to 0 bytes!**
`'a'`	End of File (EOF)	Created automatically if absent	No truncation; all writes append
`'r+'`	Beginning (offset 0)	**File must exist**	Overwrites in-place from pointer
`'w+'`	Beginning (offset 0)	Created automatically if absent	**Truncates existing file to 0 bytes!**
`'a+'`	End of File (EOF)	Created automatically if absent	Writes append; reading requires `seek(0)`

2. The Context Manager Protocol (`with` Statement)

Always open files using `with open(...) as f:`. It implements the context management protocol (`__enter__` and `__exit__`), guaranteeing deterministic file handle cleanup and operating system descriptor release even if unhandled runtime exceptions occur!

```
# File Pointer Manipulation with seek() and tell()
with open("notes.txt", "w+") as f:
    f.write("Python Master Notes")
    print("Offset after writing:", f.tell())  # 19 bytes
    f.seek(0)                                 # Reposition cursor to beginning
    content = f.read()
    print("Read Content:", content)
```

2. OOP, Dunder Methods, Operator Overloading & MRO

1. Classes, Objects & The `self` Parameter

- **Class:** A user-defined structural blueprint bundling data attributes and operational methods.
- **Object:** A concrete heap instance of a class.
- **`self` :** An explicit first parameter in instance methods representing the calling object instance (`obj.method()` ≡ `Class.method(obj)`).
- **Class Variables vs Instance Variables:** Class variables are defined in the class body and shared across all instances. Instance variables are bound to `self.var` inside `__init__` and are unique per object.

2. Magic (Dunder) Methods & Operator Overloading

```
class ComplexNum:
    def __init__(self, real, imag):
        self.real = real
        self.imag = imag

    def __add__(self, other):
        # Overloads the '+' operator
        return ComplexNum(self.real + other.real, self.imag + other.imag)

    def __str__(self):
        # Overloads print() and str()
        return f"{self.real} + {self.imag}j"

c1 = ComplexNum(3, 4)
c2 = ComplexNum(1, 2)
c3 = c1 + c2  # Invokes c1.__add__(c2)
print("Resultant Sum:", c3)  # Output: 4 + 6j
```

3. Multiple Inheritance, MRO & The Diamond Problem

When a class inherits from multiple parents with overlapping method names, Python eliminates ambiguity using the **C3 Linearization Algorithm (Method Resolution Order - MRO)**:

```
class A:
    def action(self): print("Base A")

class B(A):
    def action(self): print("Derived B")

class C(A):
    def action(self): print("Derived C")

class D(B, C):
    pass

d = D()
d.action()  # Prints: "Derived B" because B precedes C in declaration!
print([cls.__name__ for cls in D.__mro__])
# Output: ['D', 'B', 'C', 'A', 'object']
```

⚡ **KEY TAKEAWAY:**
KEY TAKEAWAY & BOTTOM LINE:

> **Composition over Inheritance:** Inheritance models an "Is-A" relationship (tight coupling, fragile base class problem). Composition models a "Has-A" relationship (a class stores references to other objects, loose coupling, higher testability).